

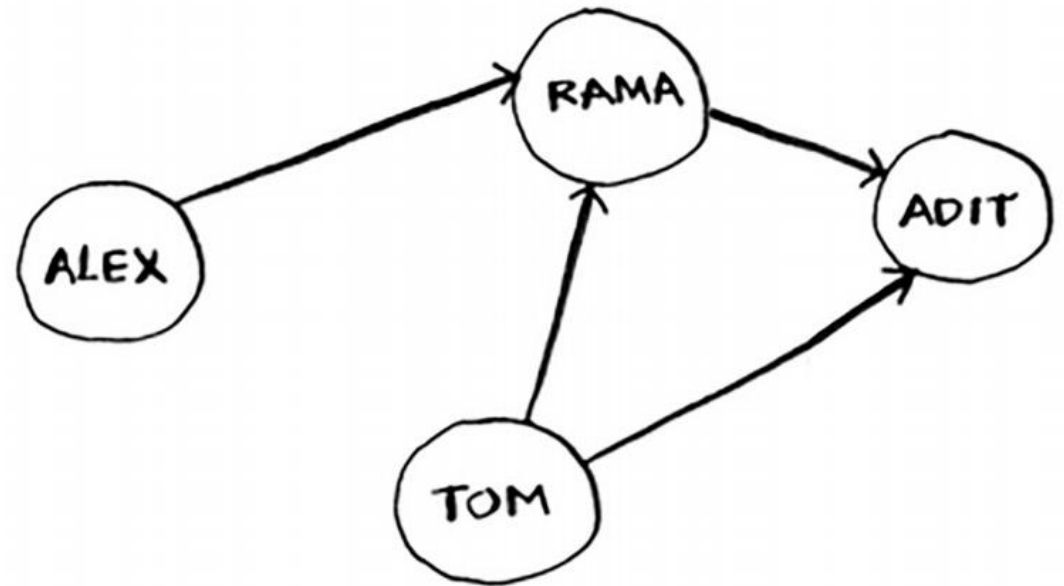
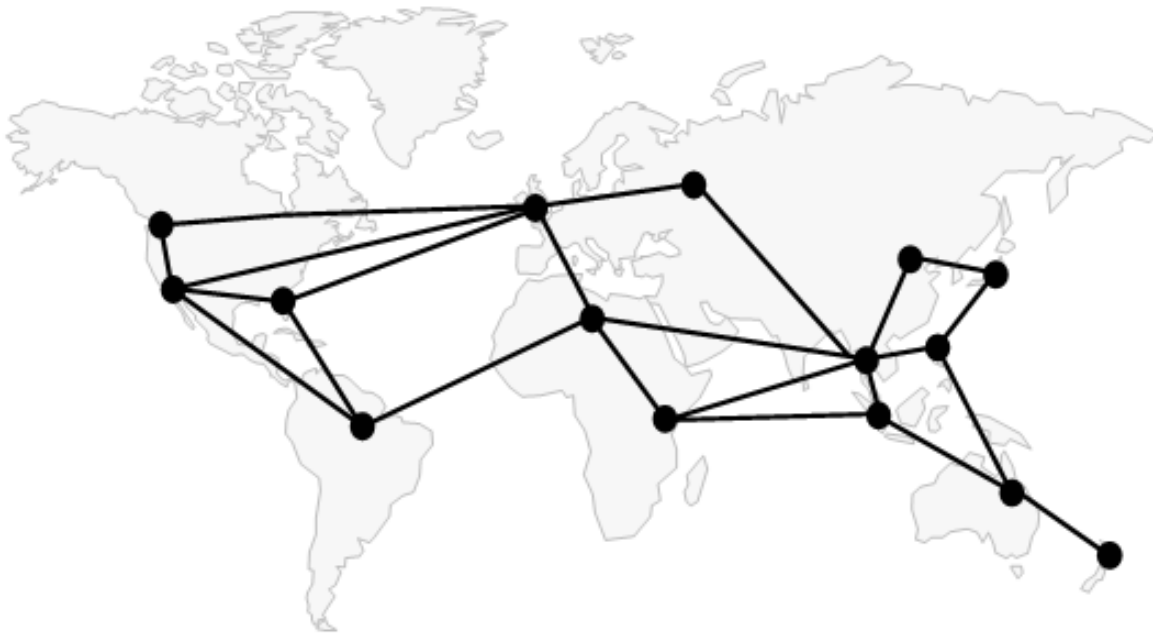


ค่ายอบรมคอมพิวเตอร์โอลิมปิก
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

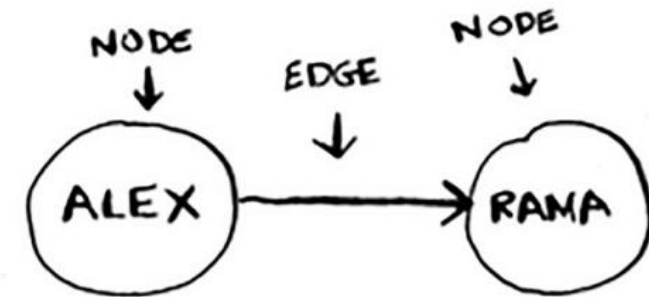
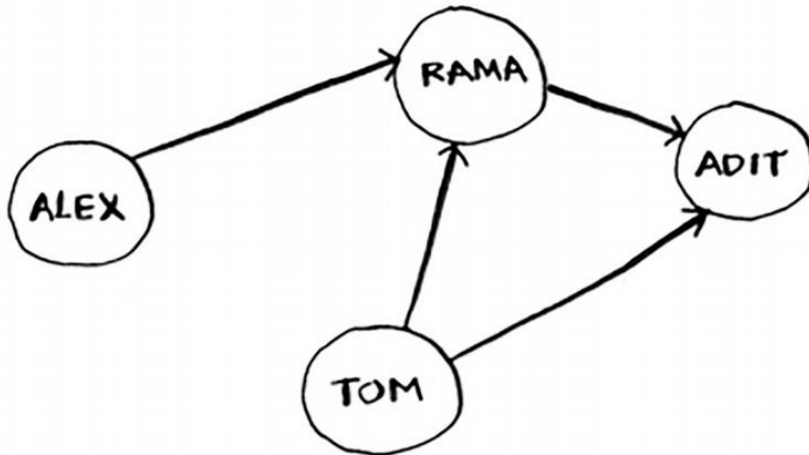
โครงสร้างข้อมูลกราฟและการสำรวจ (Graph data structure and Exploration)

อ.ลือพล พิพานเมฆาภรณ์
luepol.p@sci.kmutnb.ac.th

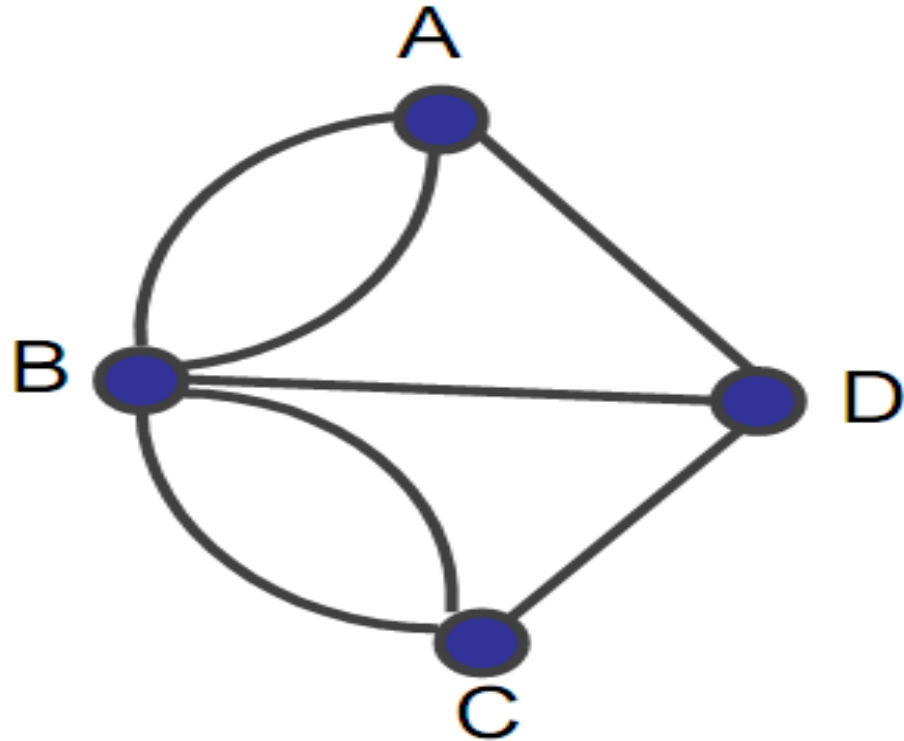
- กราฟ (graph) เป็นโครงสร้างข้อมูลซึ่งแทนเซตของความสัมพันธ์ระหว่างข้อมูล หรือเอนทิตี (entity) ที่สนใจ
- กราฟประกอบไปด้วยเซตของเวอร์เท็กซ์ (vertex) และเซตของเอจด์ (edge)



- กราฟ $G = (V, E)$ ถูกนิยามว่าเป็นเซตของเวอร์เท็กซ์ (vertex/vertices) และเอดจ์ (edge) โดยที่
 - เวอร์เท็กซ์ ทำหน้าที่แทนเอนทิตี
 - เอดจ์ คือความสัมพันธ์ระหว่างเวอร์เท็กซ์

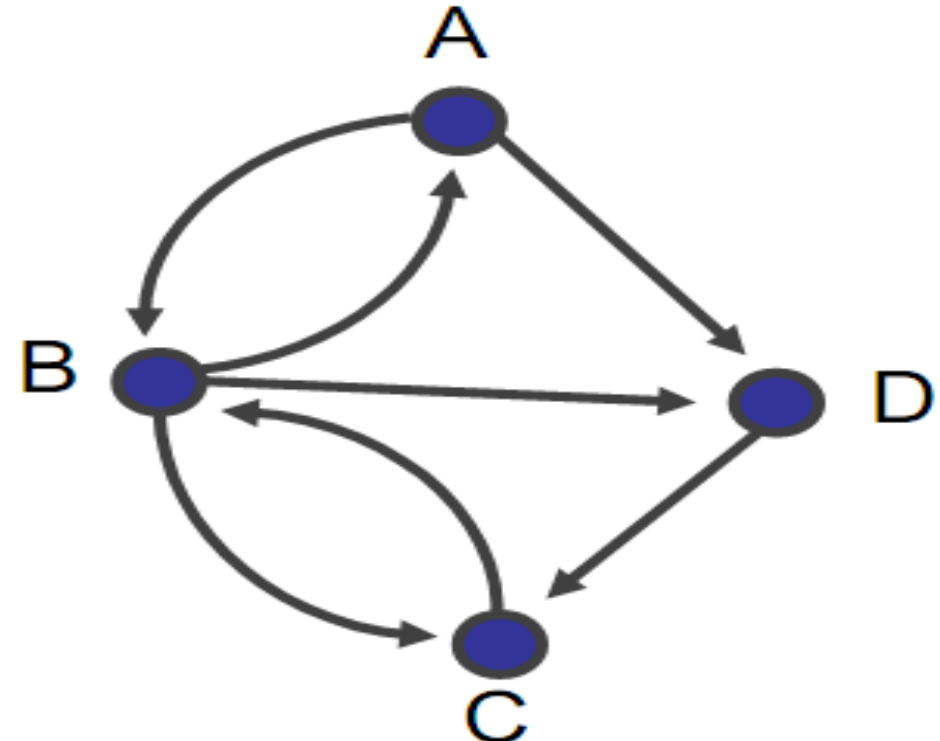


กราฟแบบไม่มีทิศทางและมีทิศทาง



เวอร์เท็กซ์: {A, B, C, D}

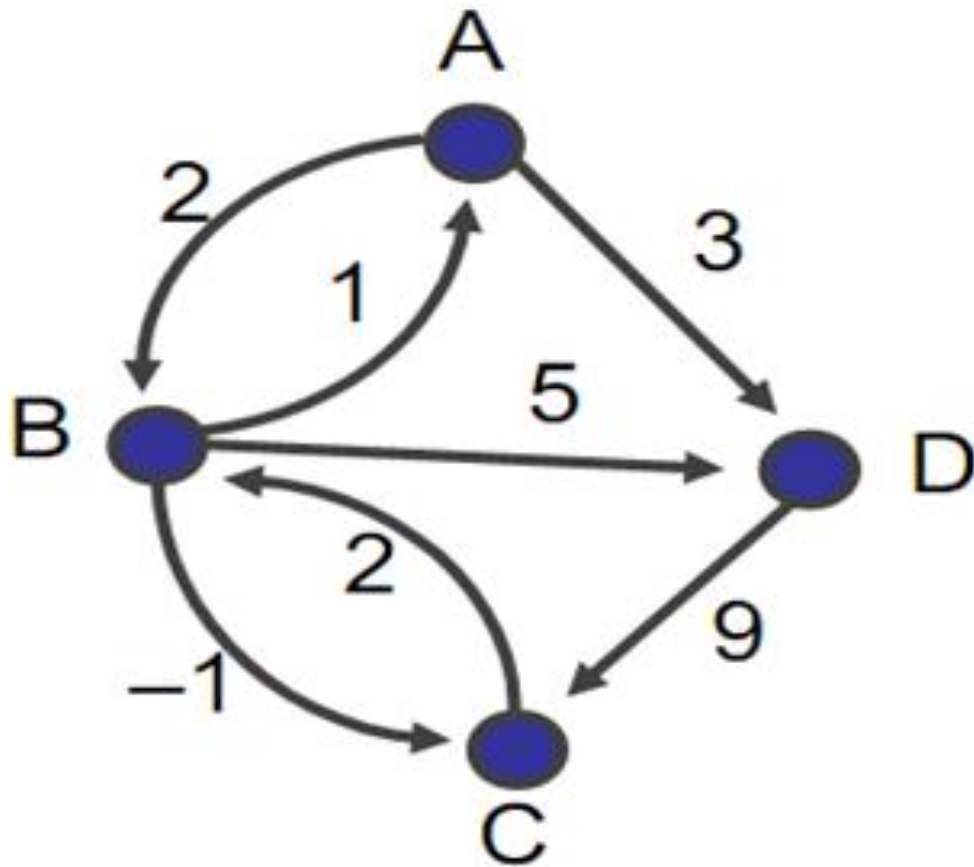
เอดจ์: {(A,B), (B,A), (B,D), (D,B), (C,D), (D,C),
(B,C), (C,B), (A,D), (D,A)}



เวอร์เท็กซ์: {A, B, C, D}

เอดจ์: {(A,B), (B,A), (A,D), (B,D), (D,C), (B,D),
(B,C), (C,B)}

กราฟแบบมีน้ำหนัก (weighted graph)



เวอร์เทกซ์: {A, B, C, D }

เอดจ์: { (A, B, 2), (B, A, 1), (A, D, 3), (D, C, 9),
(B, D, 5), (B, C, -1), (C, B, 2) }



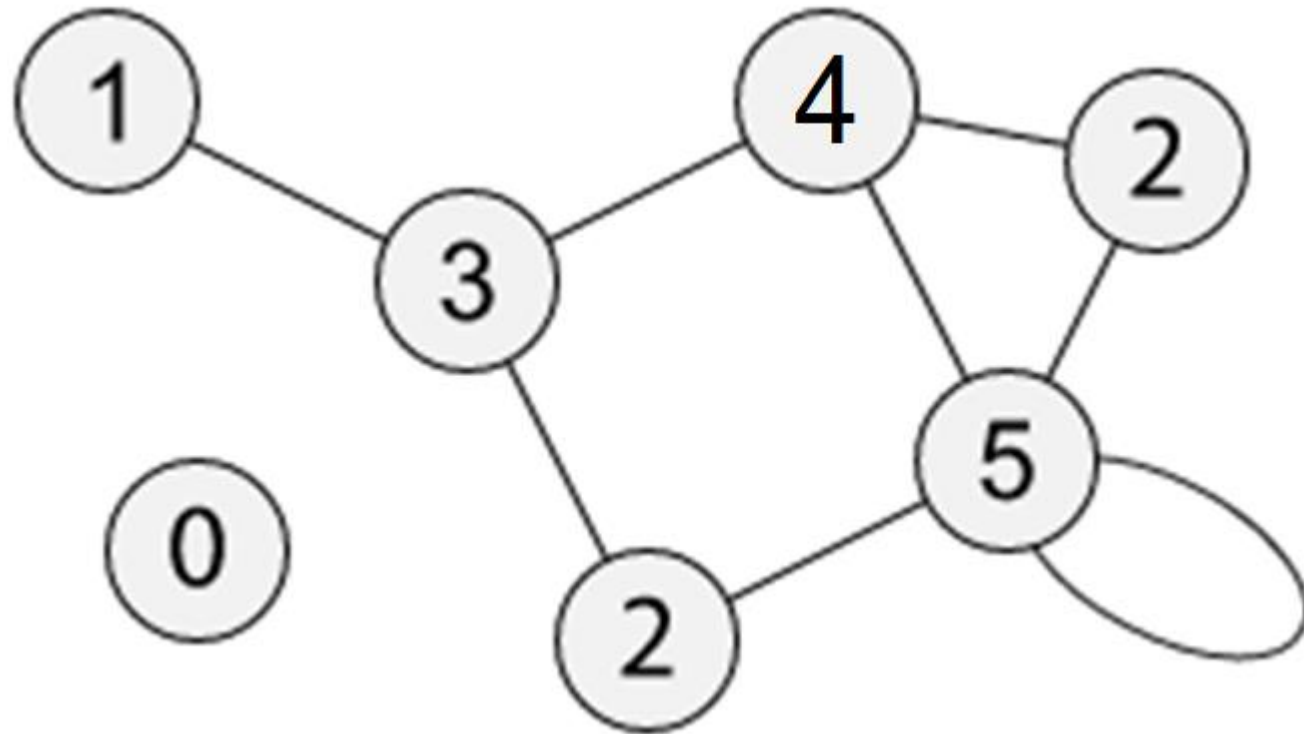
Graph terminology

- ดีกรี (degree)
- เส้นทาง (path)
- วงวน (cycle)
- ต้นไม้ Spanning tree หรือ acyclic graph
- คอมโพเนนต์ (Component)

ดีกรี (degree)

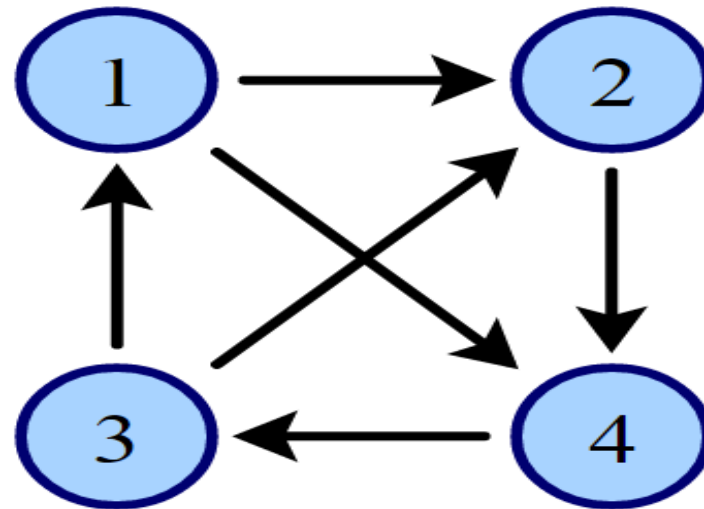
- ดีกรีของเวอร์เทกซ์ v ($\text{degree}(v)$) คือจำนวนเอจด์ซึ่งรวมกันกับ v

ดีกรีของเวอร์เทกซ์ 0 $\text{degree}(0) = 0$
 ดีกรีของเวอร์เทกซ์ 1 $\text{degree}(1) = 1$
 ดีกรีของเวอร์เทกซ์ 5 $\text{degree}(5) = 5$



ดีกรีเข้าและดีกรีออก (in-degree & out-degree)

สำหรับกราฟแบบมีทิศทาง จะมีดีกรีอยู่ 2 ชนิด คือ In-degree และ Out-degree



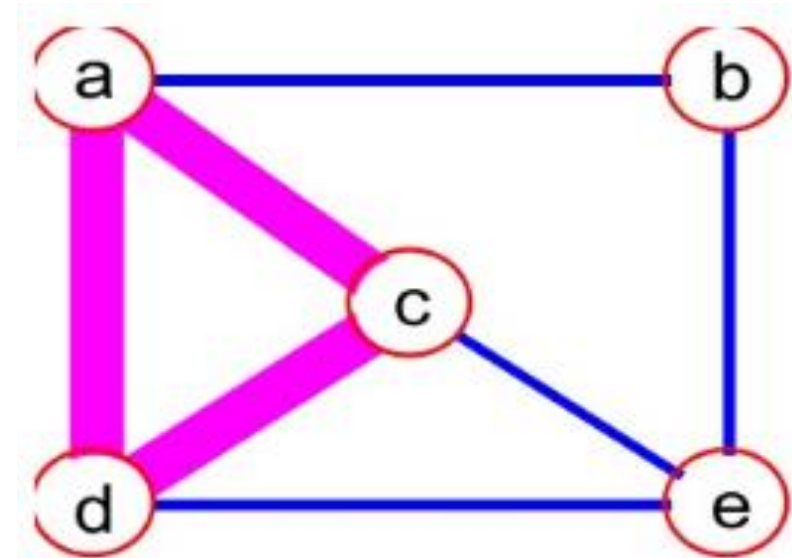
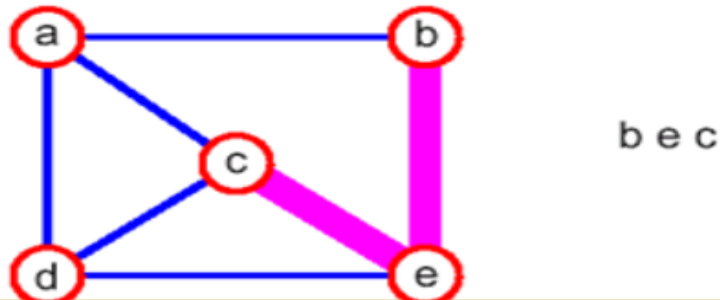
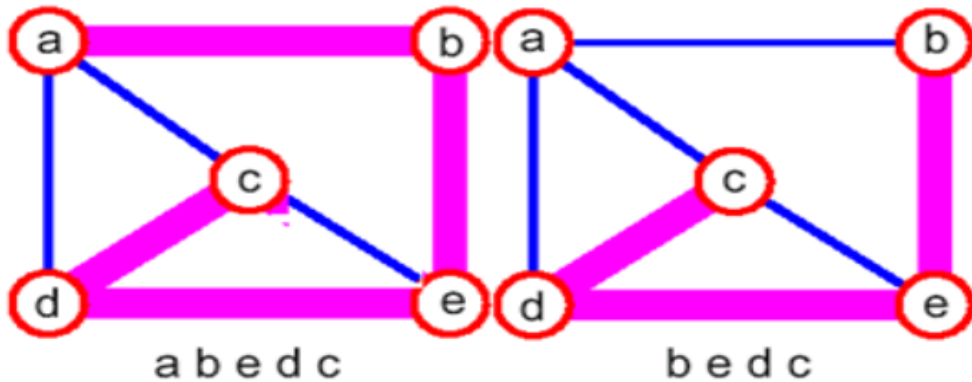
เวอร์เท็กซ์ 1 มี in-degree เท่ากับ 1 แต่มี out-degree เท่ากับ 2

เวอร์เท็กซ์ 2 มี in-degree เท่ากับ 2 แต่มี out-degree เท่ากับ 1

$$\deg(v) = \deg^-(v) + \deg^+(v)$$

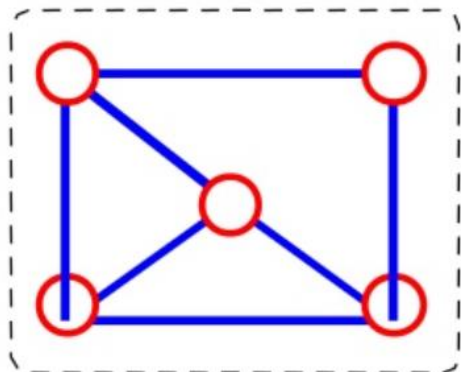
เส้นทาง (path) และวงวน (cycle)

- Path คือลำดับ (Sequence) ของ vertices ที่เชื่อมด้วยเอดจ์เดียวกัน
 - ความยาวของ path คือจำนวนเอดจ์ที่เชื่อมระหว่างเวอร์เท็กซ์เริ่มต้นและเวอร์เท็กซ์สิ้นสุด
- Cycle คือ path ซึ่งลากผ่าน vertices โดยมีเวอร์เท็กซ์เริ่มต้นและเวอร์เท็กซ์สิ้นสุดเป็นเวอร์เท็กซ์เดียวกัน

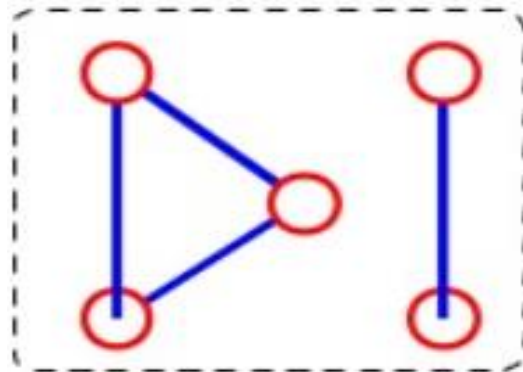


Connected components

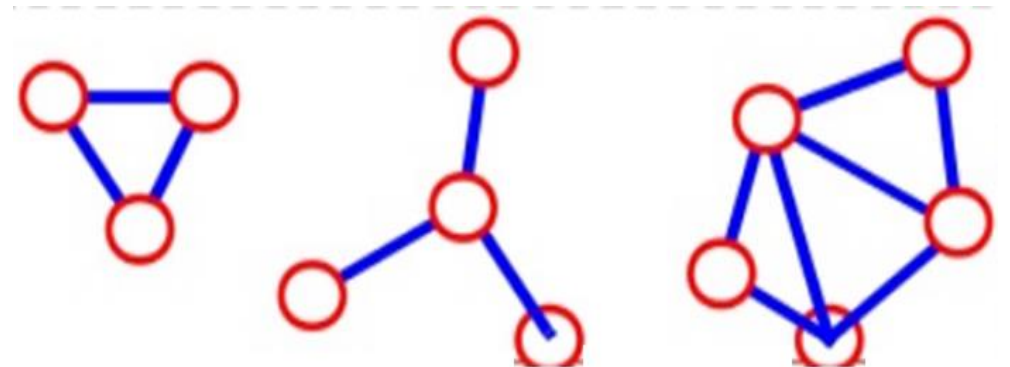
- connected graph คือกราฟที่มีเอดจ์เชื่อมโยงในทุกเวอร์เท็กซ์
- subgraph คือเซตย่อยของเวอร์เท็กซ์และเอดจ์ในกราฟ
- connected component คือ connected subgraphs ในกราฟ



connected



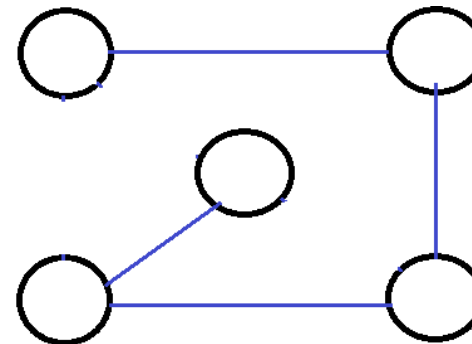
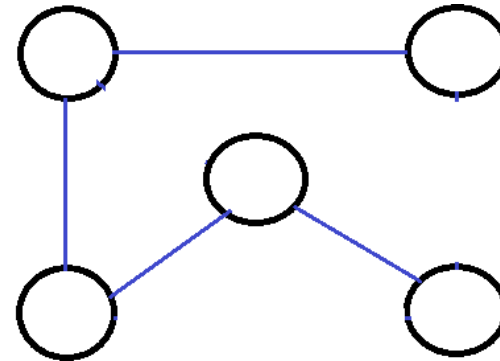
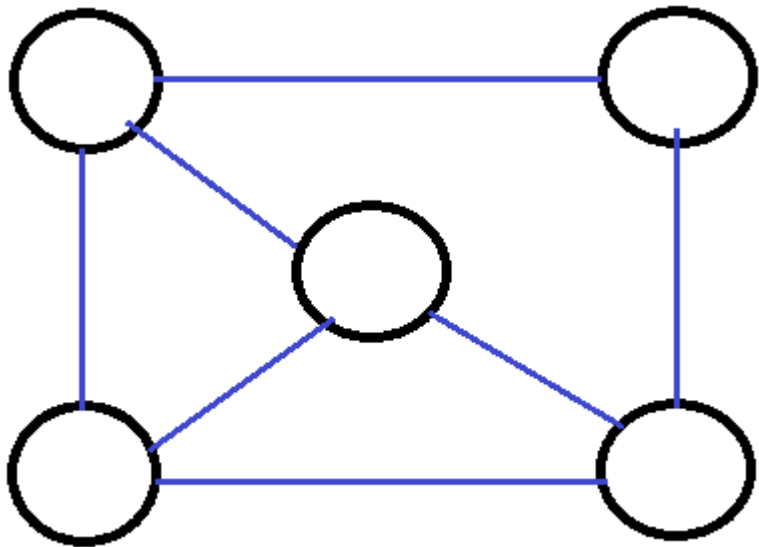
not connected



Connected components

Spanning tree

- Spanning tree คือ connected graph ซึ่งไม่มีวงวน (cycle)



- เมตริกซ์ประชิด (Adjacency Matrix)
 - มักใช้อาร์เรย์ 2 มิติ แทนเวอร์เท็กซ์ที่เชื่อมต่อกันด้วยเอดจ์
 - ข้อดีคือจัดการง่ายและทำงานได้รวดเร็ว
 - ข้อเสียคือสิ้นเปลืองหน่วยความจำ โดยเฉพาะอย่างยิ่ง sparse graph
- ลิสต์ประชิด (Adjacency List)
 - ใช้ลิสต์ร่วมกับอาร์เรย์ขนาด 1 มิติเพื่อกำหนดระหว่างเวอร์เท็กซ์
 - ข้อดีคือประหยัดหน่วยความจำที่ใช้ในการแสดงกราฟขนาดใหญ่
 - ข้อเสียคือยุ่งยากในการจัดการกว่าเมตริกซ์ประชิด

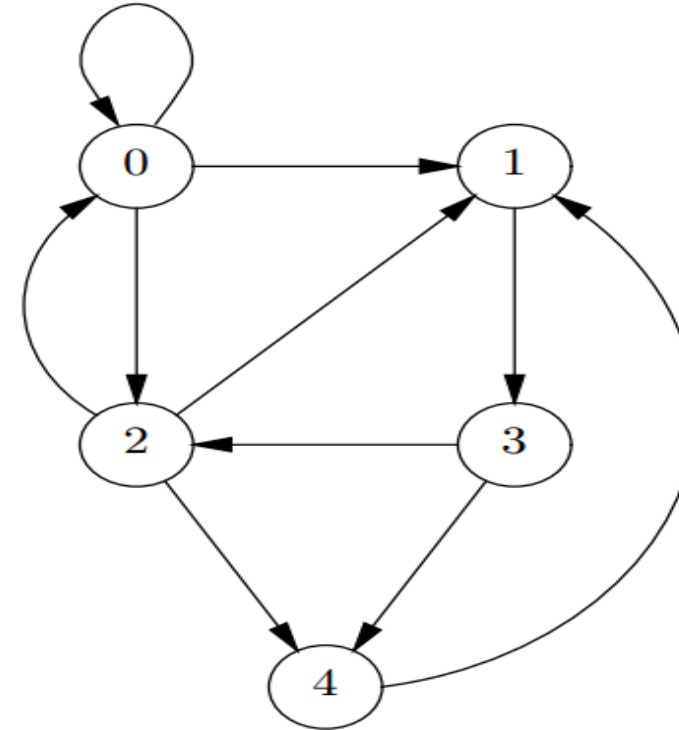
เมตริกประชิด (Adjacency matrix)

เวอร์เท็กซ์

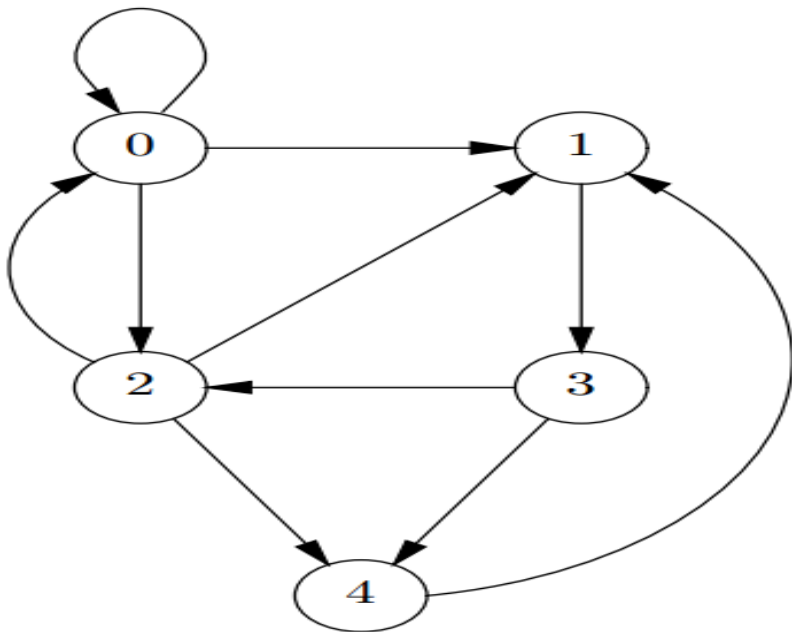
	0	1	2	3	4
0	1	1	1	0	0
1	0	0	0	1	0
2	1	1	0	0	1
3	0	0	1	0	1
4	0	1	0	0	0

เวอร์เท็กซ์

```
int edges[5][5];
```



เมตริกประชิด (Adjacency matrix)



```

#include<stdio.h>

#define MAXV 5 |

typedef struct {
    int edges[MAXV][MAXV];
    int degree[MAV];
    int nvertices, nedges;
} graph;

main()
{
    graph g;
    g.edges[0][0] = 1;      g.edges[0][1] = 1;
    g.edges[0][2] = 1;      g.edges[0][3] = 0;
    g.edges[0][4] = 0;

    g.edges[1][0] = 0;      g.edges[1][1] = 0;
    g.edges[1][2] = 0;      g.edges[1][3] = 1;
    g.edges[1][4] = 0;

}
  
```



```
void initial_graph(graph *g)
{   int i, j;
    g->nvertices = 0;
    g->nedges = 0;
    for(int i=0; i< MAXV; i++)
        g->degree[i] = 0;

    for(i=0; i< MAXV; i++)
        for(j=0; j<MAXV; j++)
            g->edges[i][j] = 0;
}
```

```
void read_graph(graph *g)
{   int i, m, x, y;
    initial_graph(g);

    scanf("%d %d",&(g->nvertrices), &m); // read #vertex and #edges in graph
    for(i=0; i< m ; i++)
    {   scanf("%d %d",&x, &y);           // read edge from x to y
        g->edges[x][y] = 1;
        g->nedges++;
        g->degree[x]++;
        g->degree[y]++;
    }
}
```

อ่านข้อมูลเพื่อสร้างกราฟ โดยเริ่มต้นจากรับค่าจำนวน
เวอร์เท็กซ์และจำนวนเอดจ์จากนั้นอ่านทีละเอดจ์ เพื่อ
สร้าง adjacency matrix



การเข้าถึงสมาชิกใน adjacency matrix

```
void print_graph(graph *g)
{ int i, j;
  for(i=0; i < g->nvertrices; i++)
  { printf("%d :", i);
    for(j=0; j<= g->nvertrices; j++)
      if(g->edges[i][j] == 1)
        printf(" %d", j);
    printf("\n");
  }
}
```

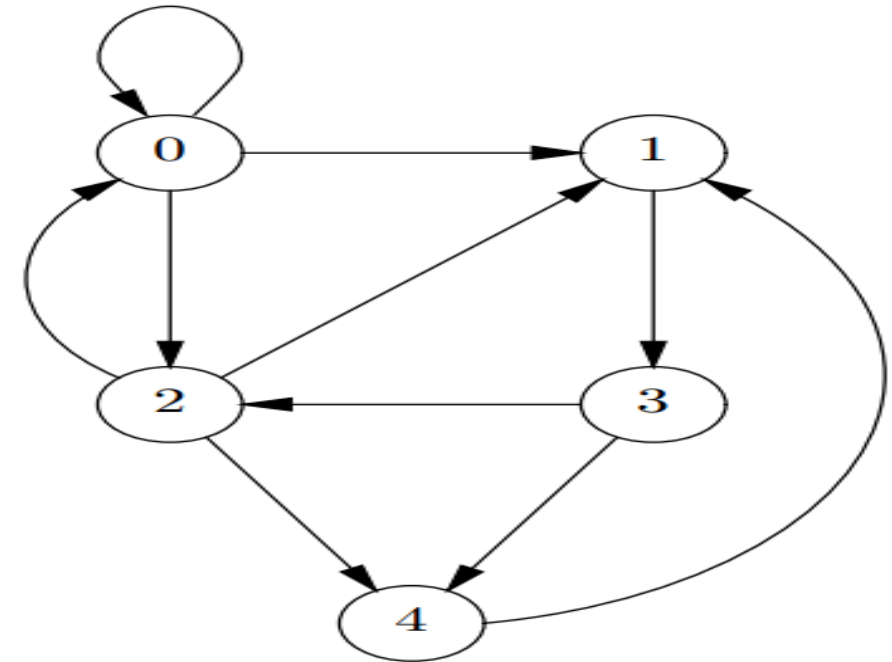
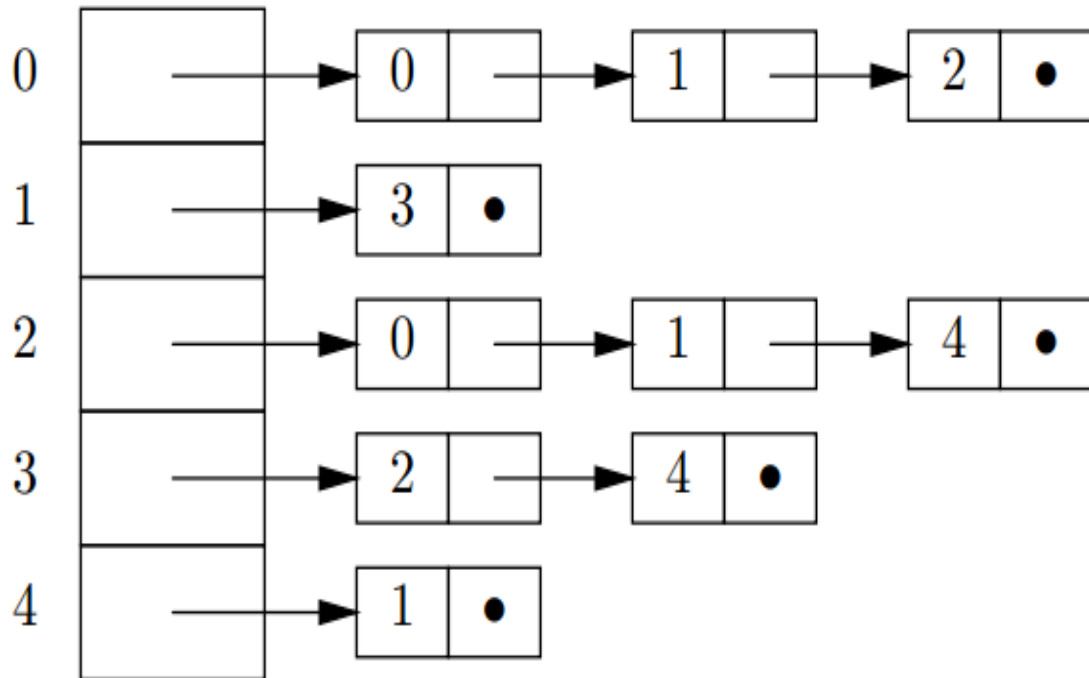
```
1 1 1 0 0
0 0 0 1 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

```
Process returned 0 (0x0)   execution time : 0.022 s
Press any key to continue.
```

ลิสต์ประชิด (adjacency list)

ประกอบไปด้วยอาร์เรย์ของลิสต์ของเอจด์ที่เชื่อมโยงในแต่ละเวอร์เทกซ์ของกราฟ

successors





โครงสร้างข้อมูลของลิสต์ประชิด

```
struct Node {  
    int item;  
    struct Node *next;  
};
```

ลิสต์ของเวอร์เทกซ์ที่เชื่อมโยง

```
struct adj_list{  
    struct Node *head;  
};
```

พอยน์เตอร์ของลิสต์

```
struct Graph {  
    int nvertices;  
    struct adj_list array[MAXV];  
};
```

อาร์เรย์ของแต่ละลิสต์



การจองหน่วยความจำสำหรับลิสต์ประชิด

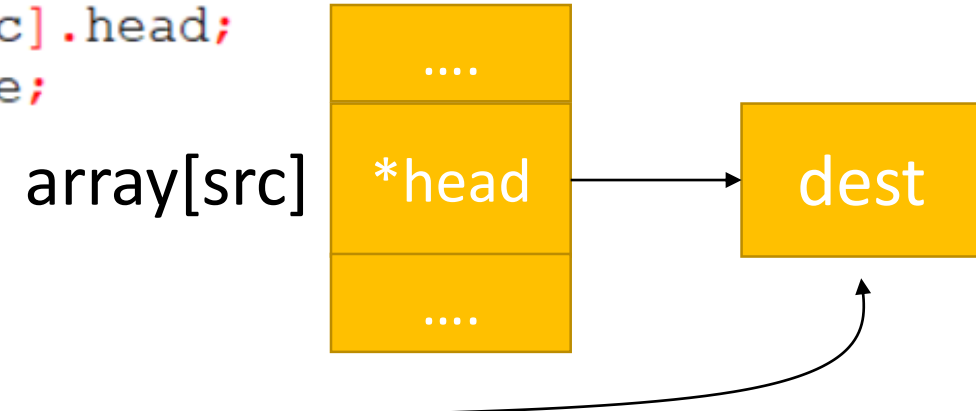
```
struct Graph *createGraph(int n)
{ struct Graph* graph = (struct Graph *) malloc(sizeof(struct Graph));

  for(int i = 0; i < n; ++i)
    graph->array[i].head = NULL;

  return graph;
}
```

การเพิ่มเอดจ์ในลิสต์ประชิด

```
void addEdge(struct Graph* graph, int src, int dest)
{ struct Node* newnode = CreateNode(dest);
  newnode->next = graph->array[src].head;
  graph->array[src].head = newnode;
}
```



```
struct Node *CreateNode(int dest)
{ struct Node *newnode = (struct Node *) malloc(sizeof(struct Node));
  newnode->item = dest;
  newnode->next = NULL;

  return newnode;
}
```



การเข้าถึงสมาชิกในลิสต์ประชิด

```
void printGraph(struct Graph *g)
{ int v;

  for(v=0; v < g->nvertices; ++v) {
    struct Node* p = g->array[v].head;
    printf("%d", v);
    while(p)
      { printf(" -> %d", p->item);
        p = p->next;
      }
    printf("\n");
  }
}
```

```
0 -> 4 -> 1
1 -> 2
2 -> 3
3 -> 4
4
```

```
Process returned 0 (0x0)   execution time : 0.029 s
Press any key to continue.
```

```
int main()
{
  struct Graph *g = createGraph(MAXV);

  addEdge(g, 0, 1);
  addEdge(g, 0, 4);
  addEdge(g, 1, 2);
  addEdge(g, 2, 3);
  addEdge(g, 3, 4);

  g->nvertices=MAXV;

  printGraph(g);

  return 0;
}
```



การสำรวจกราฟ (Graph Traversal)

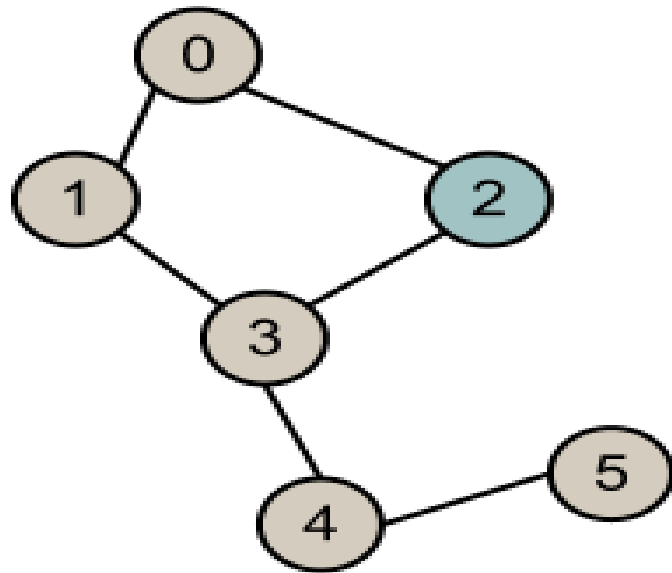
- ปัญหาพื้นฐานที่สำคัญที่สุดของกราฟ คือการสำรวจกราฟ (graph exploration)
- สองอัลกอริทึมที่นิยมใช้ในการสำรวจกราฟ
 - Breadth First Search (BFS)
 - Depth First Search (DFS)



Breadth First Search (BFS)

1. กำหนดเวอร์เทกซ์เริ่มต้น (source vertex) เป็น current vertex
2. สำรวจ neighbors ของ current vertex ซึ่งยังไม่เคยถูก visit โดยนำไปเก็บไว้ในคิว (Queue)
3. ดึง neighbors จากคิว ทำซ้ำในขั้นตอน 2
4. จบการทำงานเมื่อทุกเวอร์เทกซ์ถูก visit หรือไม่มีเวอร์เทกซ์ใดอยู่ในคิว

Breadth First Search



1. $q = \{\}$
2. $q = \{2\}$
3. $q = \{0, 3\}$
4. $q = \{1\}$
5. $q = \{4\}$
6. $q = \{5\}$

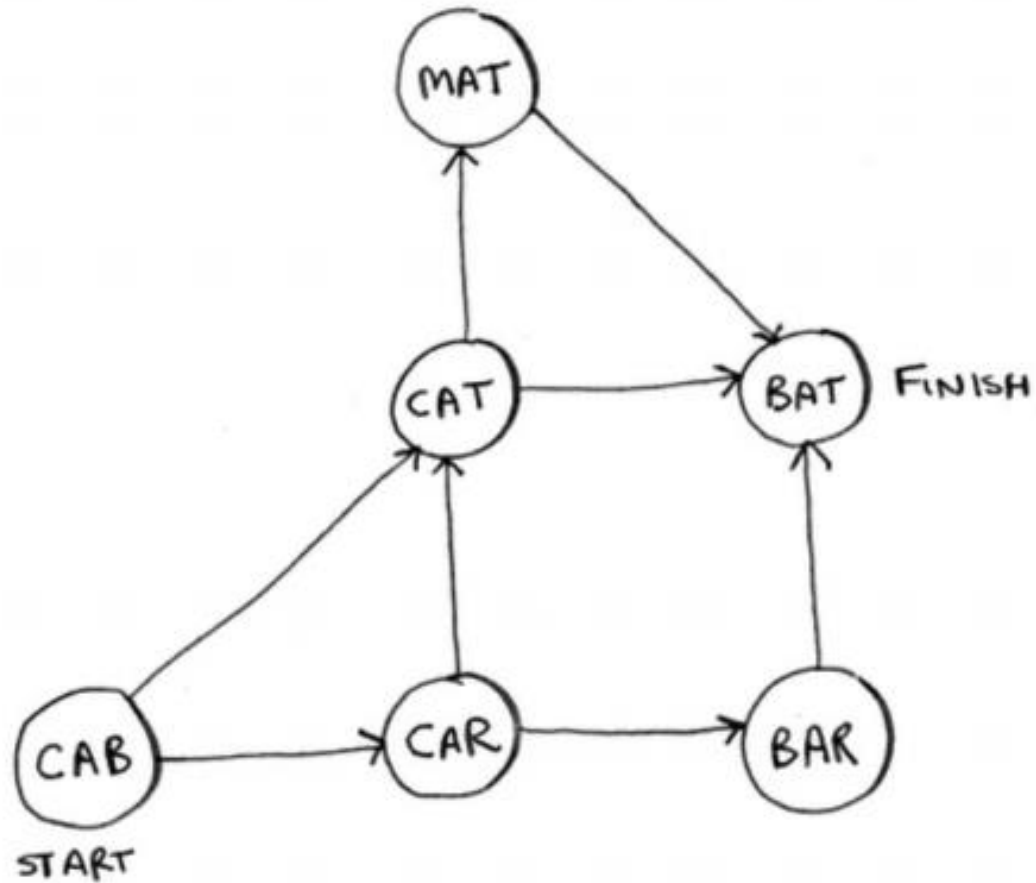
Using a queue



Implementation

```
void BFS (graph *g, int start)
{
    queue Q;                                // queue data structure
    initial_queue(&Q);                       // make empty queue
    enqueue(&Q, start);                     // enqueue with start node
    while(empty(&Q) == FALSE)
    {
        v = dequeue(&Q);                    // call dequeue
        printf("visit node : %d", v);       // print visit node
        visited[v] = TRUE;                  // mark visited v
        for(i=0; i < g->nvertrices; i++)     // explore neighbors
        {
            if(visited[i] == FALSE)
            {
                if(g->edges[i][v] == 1 || g->edges[v][i] == 1)
                {
                    enqueue(&Q, i);         // next explore
                }
            }
        }
    }
}
```

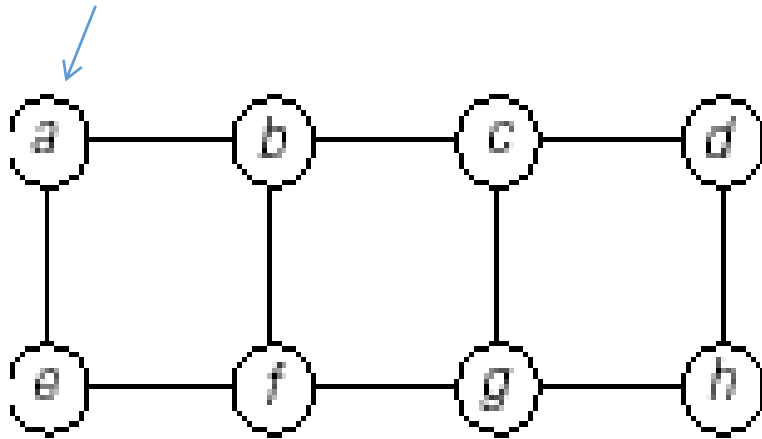
- ตรวจสอบ Connectivity ของกราฟ :
 - เนื่องจาก BFS สิ้นสุดการทำงานเมื่อทุกๆ vertex ที่เชื่อมต่อกับ vertex เริ่มต้นได้รับการพิจารณา การตรวจสอบ connectivity สามารถทำได้โดยใช้ BFS traversal ซึ่งเริ่มจาก vertex ใดก็ได้ เมื่อ algorithm สิ้นสุดการทำงานให้ตรวจสอบว่า ทุกๆ vertex ใน graph ได้รับการพิจารณาแล้วหรือไม่ graph จะถือว่าเป็น connected ถ้าทุกๆ vertex ถูก visited
- ค้นหาพาสที่สั้นที่สุด (shortest path)
 - สำรองต้นไม้ม BFS จะทำให้ได้ Path ที่มีความยาวที่สั้นที่สุดจาก source vertex ไปยังทุกๆ vertex



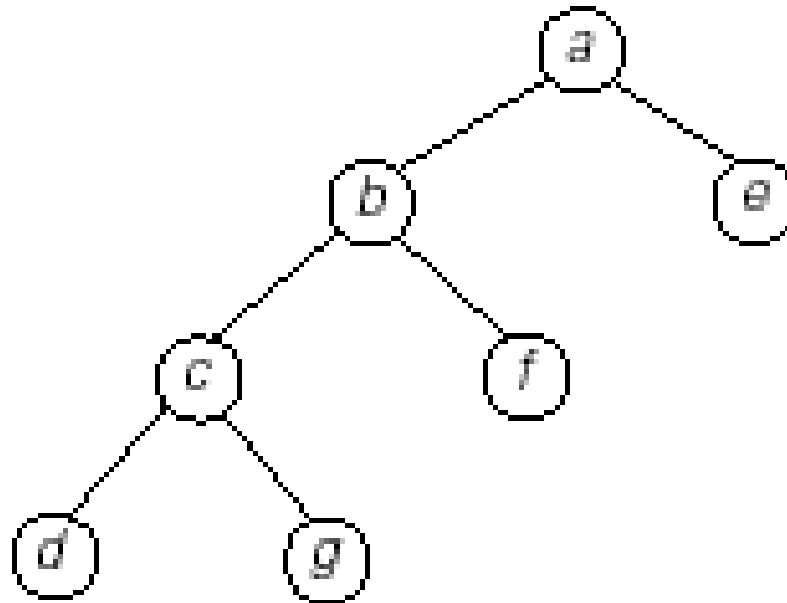
ใช้ BFS ค้นหา shortest path จาก
CAB -> BAT

ต้นไม้ BFS

Start vertex



กราฟ G



ต้นไม้ BFS

เป็นต้นไม้ที่แสดงความสัมพันธ์ระหว่าง
เวอร์เท็กซ์และ neighbor ของมัน
เมื่อสำรวจกราฟด้วย BFS

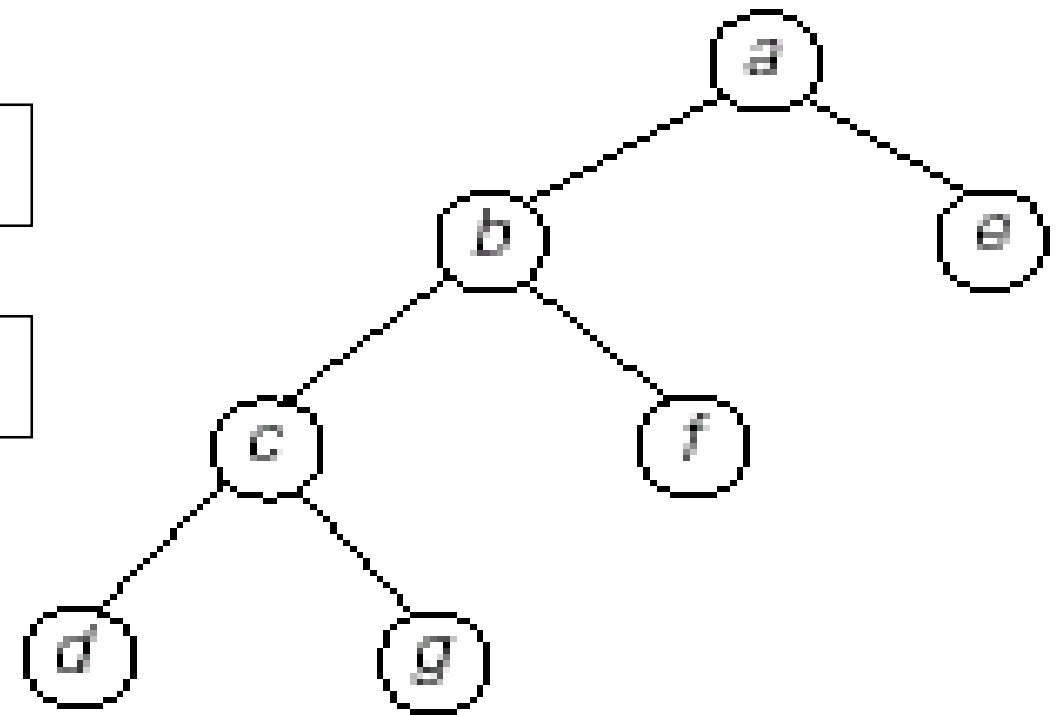
ต้นไม้ BFS ที่เกิดจากลำดับในการ
สำรวจกราฟ มีความสำคัญเช่น เช่น
การค้นหา path ที่สั้นที่สุดจากเวอร์เท็กซ์เริ่มต้นในกราฟ

Shortest Path $a \rightarrow b = 1$, $a \rightarrow e = 1$, $a \rightarrow c = 2$, $a \rightarrow d = 3$, $a \rightarrow g = 3$

ทำอย่างไรจึงจะจัดเก็บต้นไม้ BFS

Parent child array: เก็บข้อมูลเวอร์เท็กซ์ซึ่งเป็น neighbor ของเวอร์เท็กซ์ปัจจุบัน

	0	1	2	3	4	5	6
node[]	a	b	c	d	e	f	g
	0	1	2	3	4	5	6
parent[]	-1	0	1	2	0	1	2





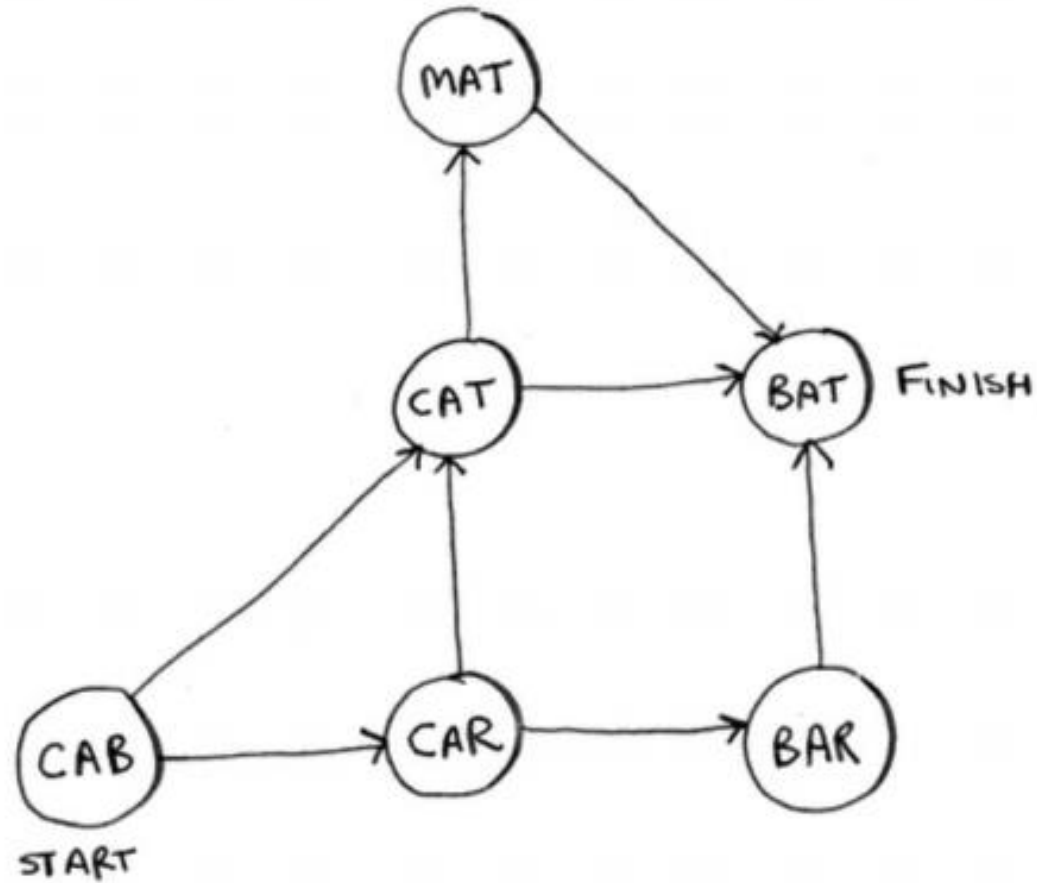
```
void BFS (graph *g, int start)
{
    queue Q;                                // queue data structure
    initial_queue(&Q);                      // make empty queue
    enqueue(&Q, start);                     // enqueue with start node
    while(empty(&Q) == FALSE)
    {
        v = dequeue(&Q);                    // call dequeue
        printf("visit node : %d", v);      // print visit node
        visited[v] = TRUE;                  // mark visited v
        for(i=0; i < g->nvertrices; i++)    // explore neighbors
        {
            if(visited[i] == FALSE)
            {
                if(g->edges[i][v] == 1 || g->edges[v][i] == 1)
                {
                    enqueue(&Q, i);         // next explore
                    parent[i] = v;
                }
            }
        }
    }
}
```



Depth First Search (DFS)

- ขั้นตอนที่ 1 กำหนด start vertex เป็น current vertex
- ขั้นตอนที่ 2 เช็ค neighbor ของ current vertex ว่าถูกสำรวจหรือไม่
- ขั้นตอนที่ 3 หาก neighbor ในขั้นตอน 2 ยังไม่เคยถูกสำรวจ จะทำการเก็บ current vertex ลงใน stack และเปลี่ยนสถานะของ neighbor ให้เป็น current vertex
- ขั้นตอนที่ 4 หากไม่มี neighbor ที่สำรวจไปได้ จะดึง vertex ใน stack ออกมาและเปลี่ยนสถานะให้กลายเป็น current vertex
- ขั้นตอนที่ 5 ทำซ้ำขั้นตอนที่ 2 จน stack ว่าง

DFS example





Implementation

```
void DFS (graph *g, int v)
{
    int i;
    printf("%d ", start);           // process visit node
    visited[v] = TRUE;              // mark visit
    for(i=0; i < g->nvertrices; i++) // explore neighbors
    {
        if(g->edges[i][v]== 1 || g->edges[v][i] == 1)
        {
            if(visit[i] == FALSE)   // if not visit
            {
                parent[i] = v;
                DFS(g, i);
            }
        }
    }
}
```